

Assignment 2 Report

Author: Riley Link Date: 9/23/2026

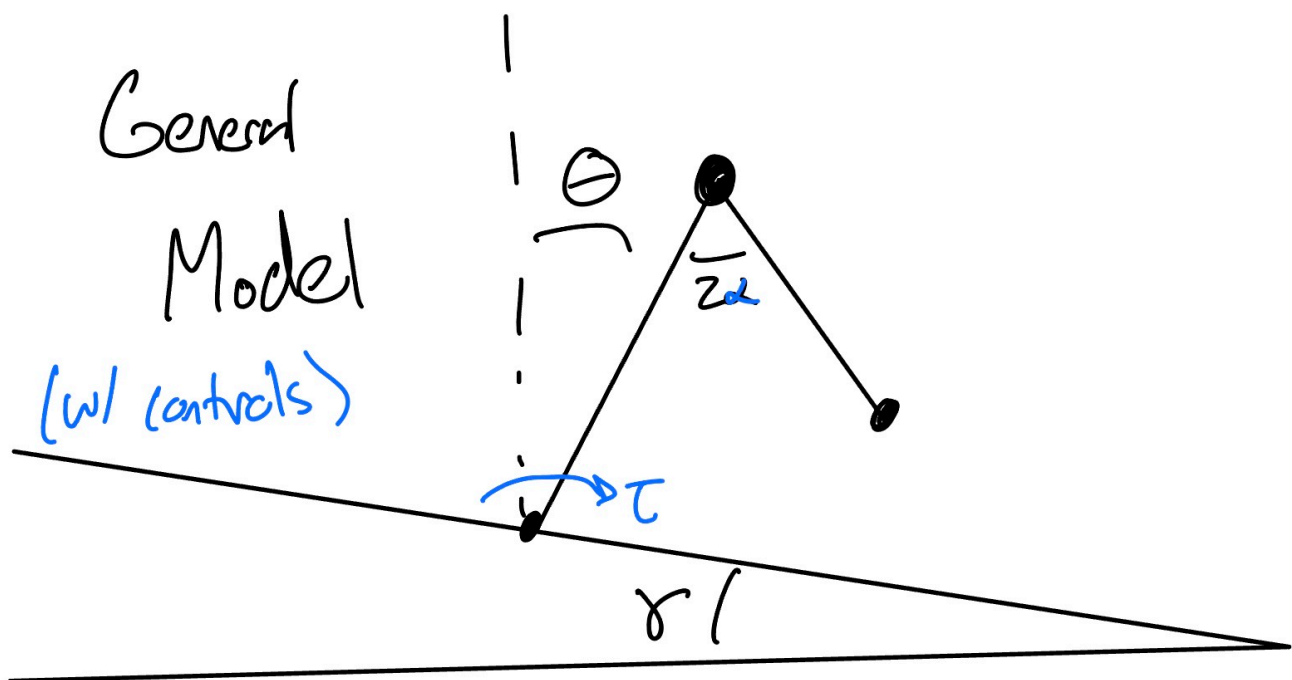
Model Implementation

In this assignment we were tasked with controlling the rimless wheel, aka the walker, so that it comes to a balancing point. To do this, we allowed a torque around the base along with full control of the angle alpha, i.e. being able to move its leg out to where we want. These two controls both had restrictive bounds that made this task difficult.

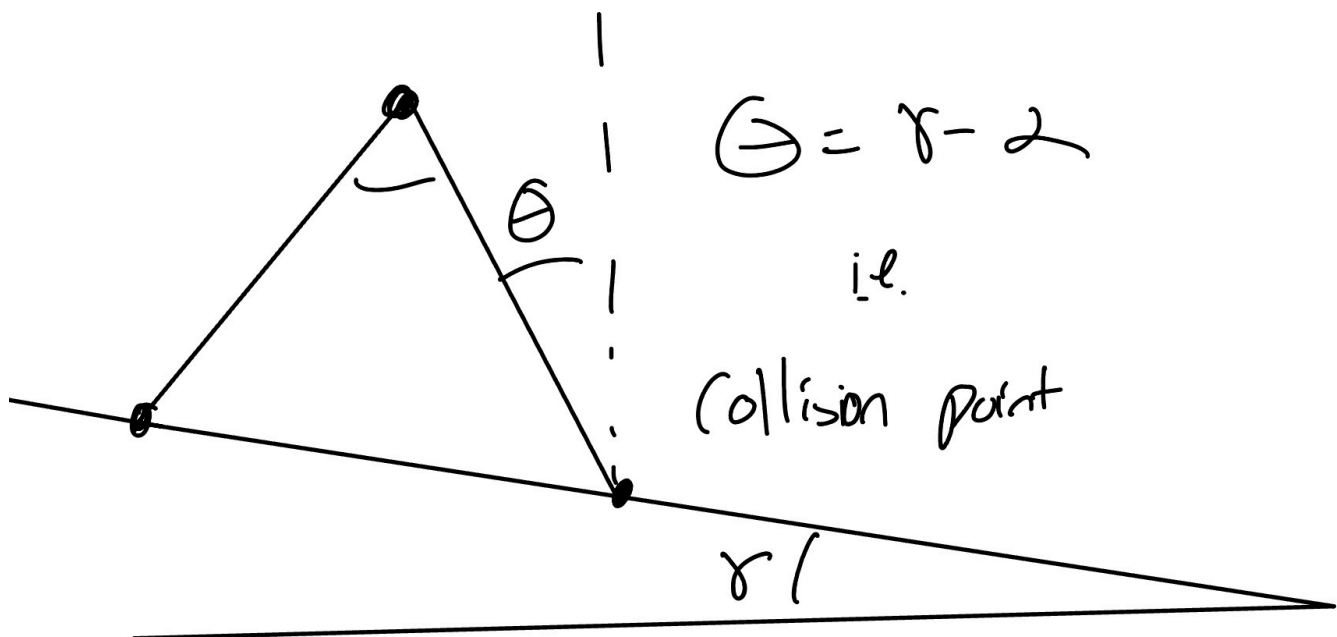
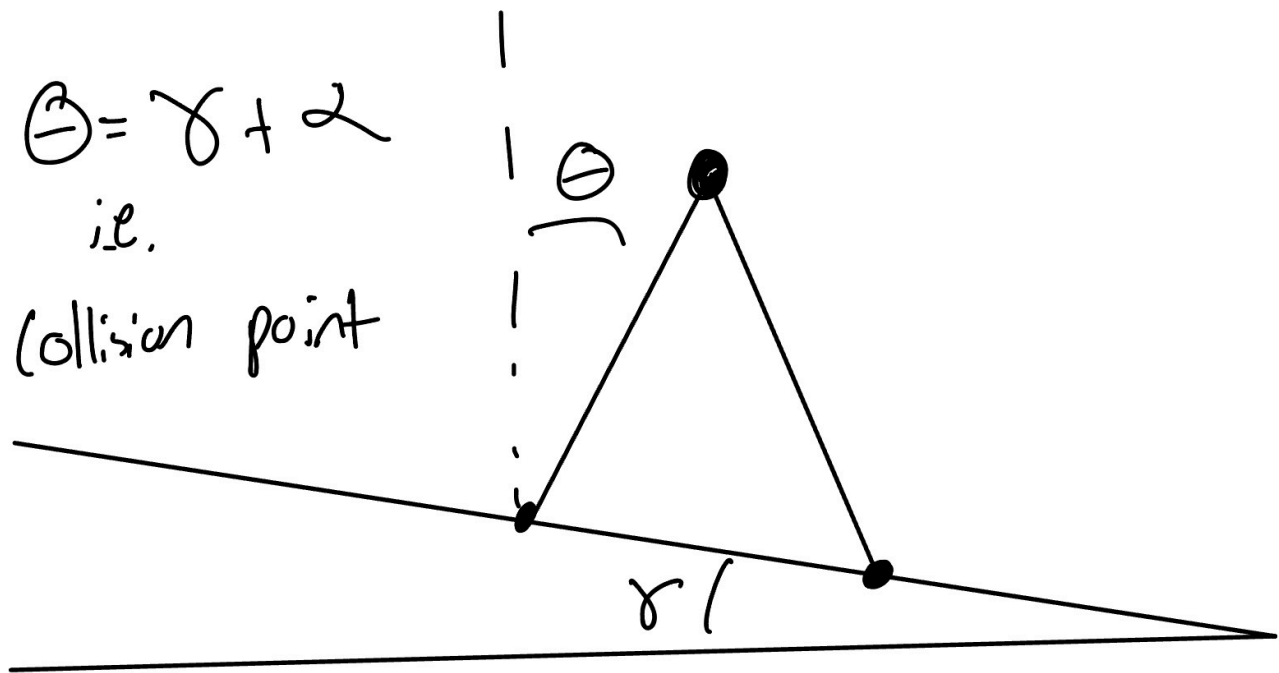
To begin, below are the dynamics of the walker, now implemented using the control torque,

$$\ddot{\theta} = \frac{g}{l} \sin(\theta) + \frac{1}{ml^2} \tau$$

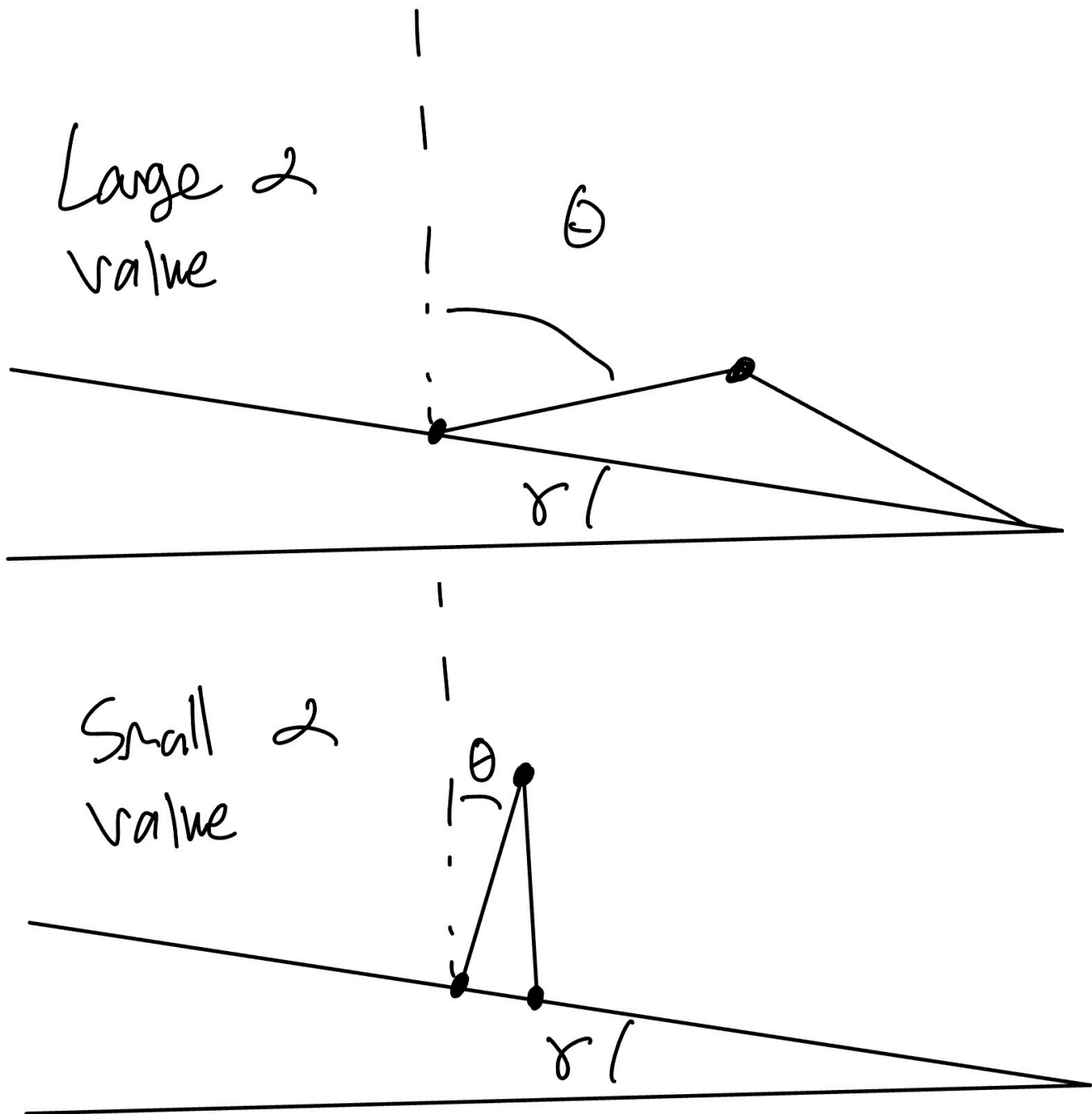
where $\tau \in [-0.1mgl, 0.05mgl]$. Here, we use the model where theta is upright with respect to the vertical inertial frame. The following sketch shows the general model:



Then collision points are as shown:



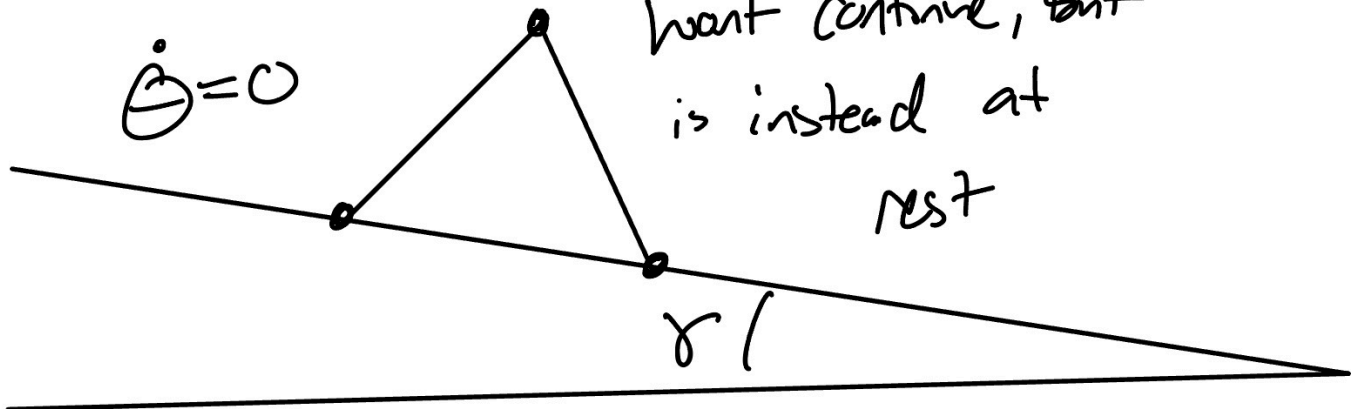
Then we can see what it looks like based on changing alpha i.e. large alpha to have wide legs or small alpha for narrow legs. Intuitively, if we have larger alpha, our steps take long and we can reach further, while also slowing down our momentum and making that collision dynamics a larger shift. Vice versa, the narrower legs keep our momentum moving and keep our velocity high.



Finally we have our failure and goal states, i.e. our failure is when we stop moving and our goal is when we get balanced perfectly:

failure mode if its stuck and
want continue, but
is instead at
rest

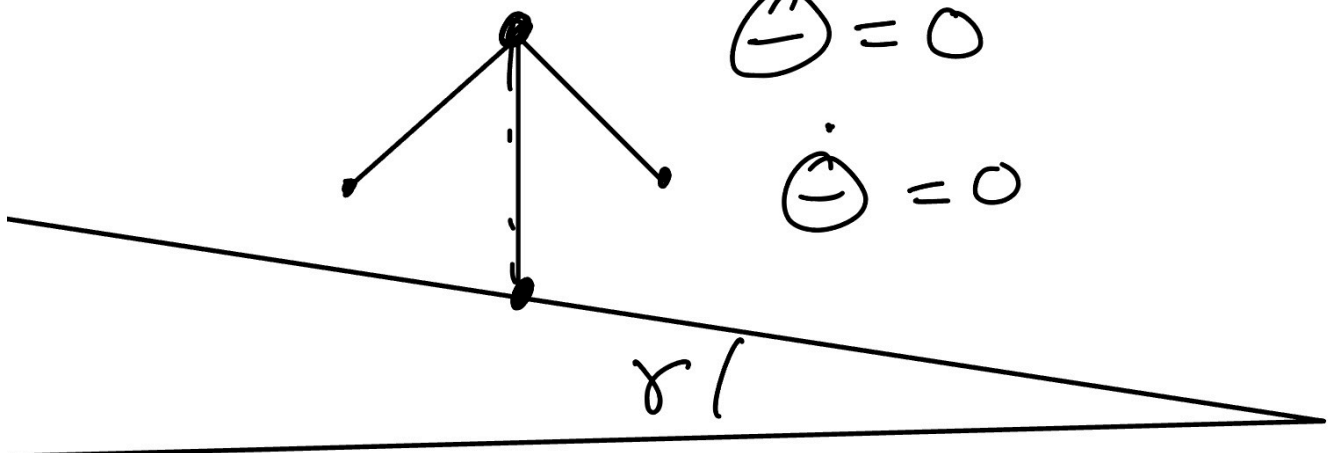
$$\dot{\theta} = 0$$



Goal

$$\theta = 0$$

$$\dot{\theta} = 0$$



Stablizing using Feedback Linearization

To stablize our controller, we used the simplest form of feedback linearization: PD control. Specifically, we implemented the function `calculate_torque(state, params)` which returned,

$$\tau = -mgl \sin(\theta) - K_p \theta - K_d \dot{\theta}.$$

Here the constants K_p and K_d determine how quickly our controller acts.

Plugging this tau function into the dynamics yields the following simplification,

$$\begin{aligned} \ddot{\theta} &= \frac{g}{l} \sin(\theta) + \frac{1}{ml^2} \tau \\ &= \frac{g}{l} \sin(\theta) + \frac{(-mgl \sin(\theta) - K_p \theta - K_d \dot{\theta})}{ml^2} \\ &= -\frac{K_p}{ml^2} \theta - \frac{K_d}{ml^2} \dot{\theta}. \end{aligned}$$

As long as we choose the constants K_p and K_d to be positive, this yields a dynamical system that sends θ and $\dot{\theta}$ to zero. In practice, we chose $K_p = K_d = 10$ as we found that it was large enough to quickly control the walker while not overshooting the equilibrium and it brings the walker to the equilibrium in a smooth manner.

HOWEVER, note that we calculate the torque using a clip function, meaning that if the requested torque is outside of our bounds we simply return the outer bound, thus never breaking our constraints.

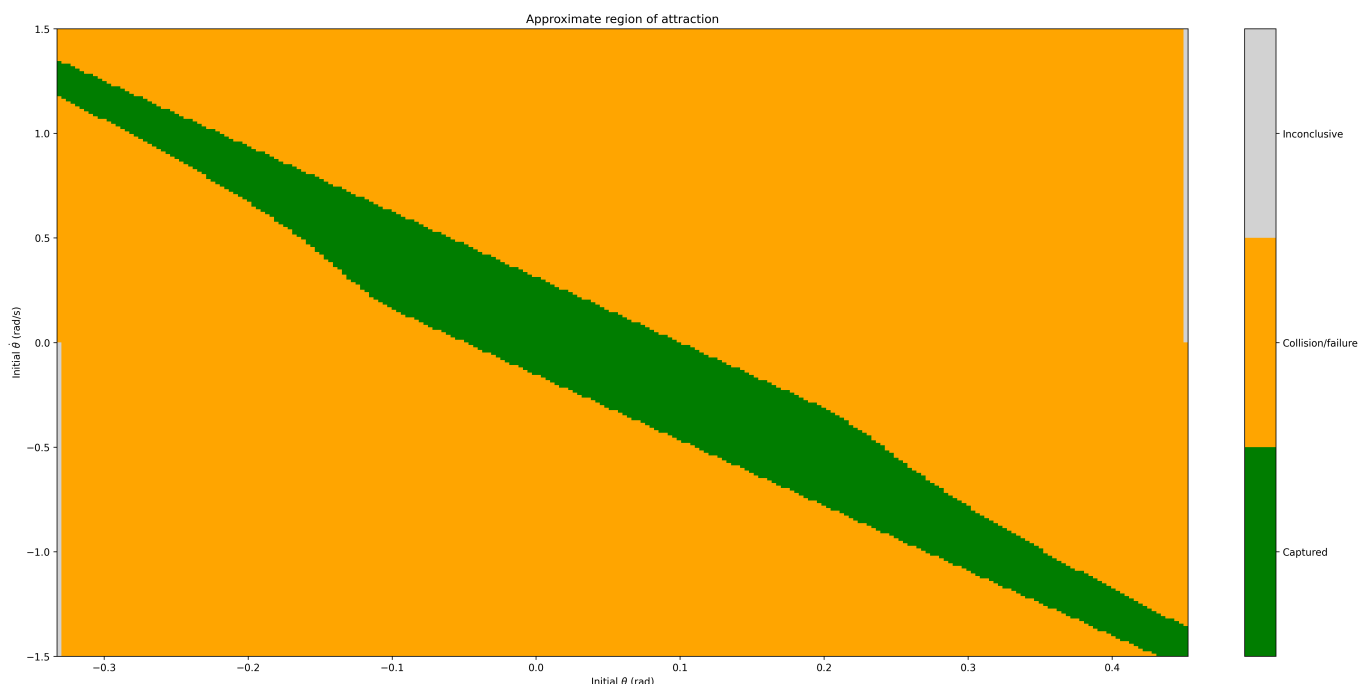
Determining the ROA

To determine the ROA, we used a similar method as in the previous assignment. Specifically, this was implemented in the function `plot_controller_roa`.

Within this function a couple major things are happening:

- First, a grid for θ and $\dot{\theta}$ values are formed via linearly spacing values across a chosen number of values. We chose $n = 250$ for both values to provide a fine grid resolution. Note that we can only choose $\theta \in [\gamma - \alpha, \gamma + \alpha]$ as those limits provide where the next leg hits the ground.
- Then, the function starts at the given initial conditions, and simulates forward in time via a runge-kutta 4 integrator while implementing the controllers torque.
- Using the event guard, if the walker takes a step this is automatically labeled as a failure to reach the ROA (as we are only considering the direct controller ROA not across all time)
- Then, we chose tolerances for both θ and $\dot{\theta}$ to say that once θ and $\dot{\theta}$ are under these tolerances (chosen to be 0.001), then we classify that initial condition as being in the controller ROA since it comes to a full stop balance
- Finally, the plot is made to include balanced, failure, and inconclusive runs (which really only account for random breaking cases like we sampled an alpha that is too far)

Below is the given controller ROA figure.



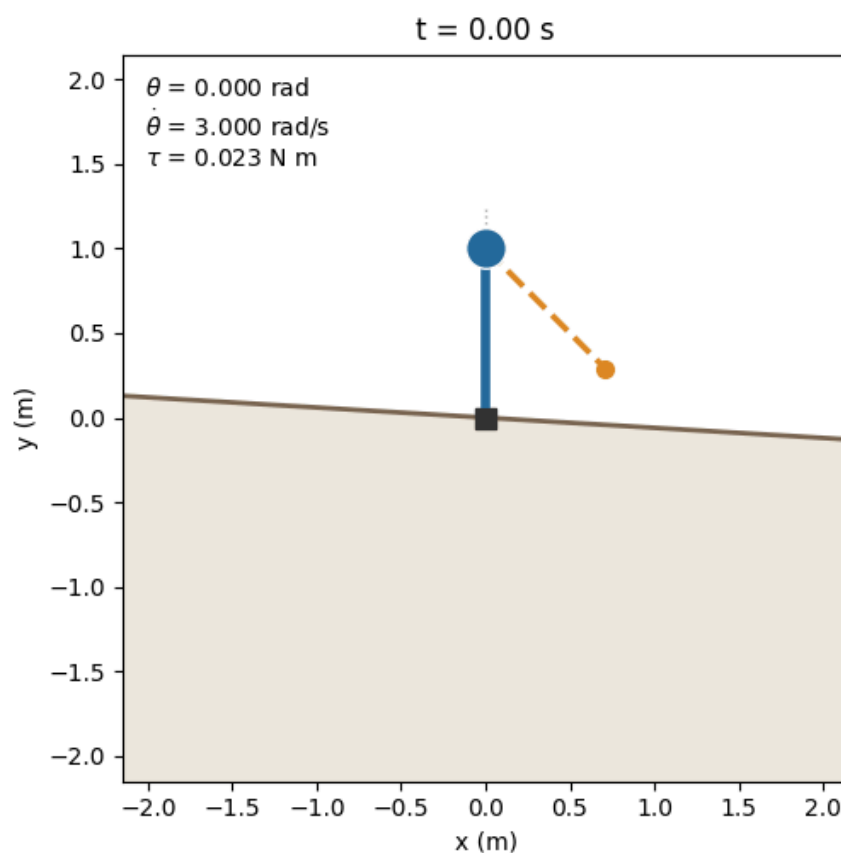
To analyze this, it is pretty simple. We expect that the further away from the zero θ equilibrium we are, we need to send it in the other direction. That is, say $\theta > 0$. Then, we need $\dot{\theta} < 0$, i.e. negative velocity, to send

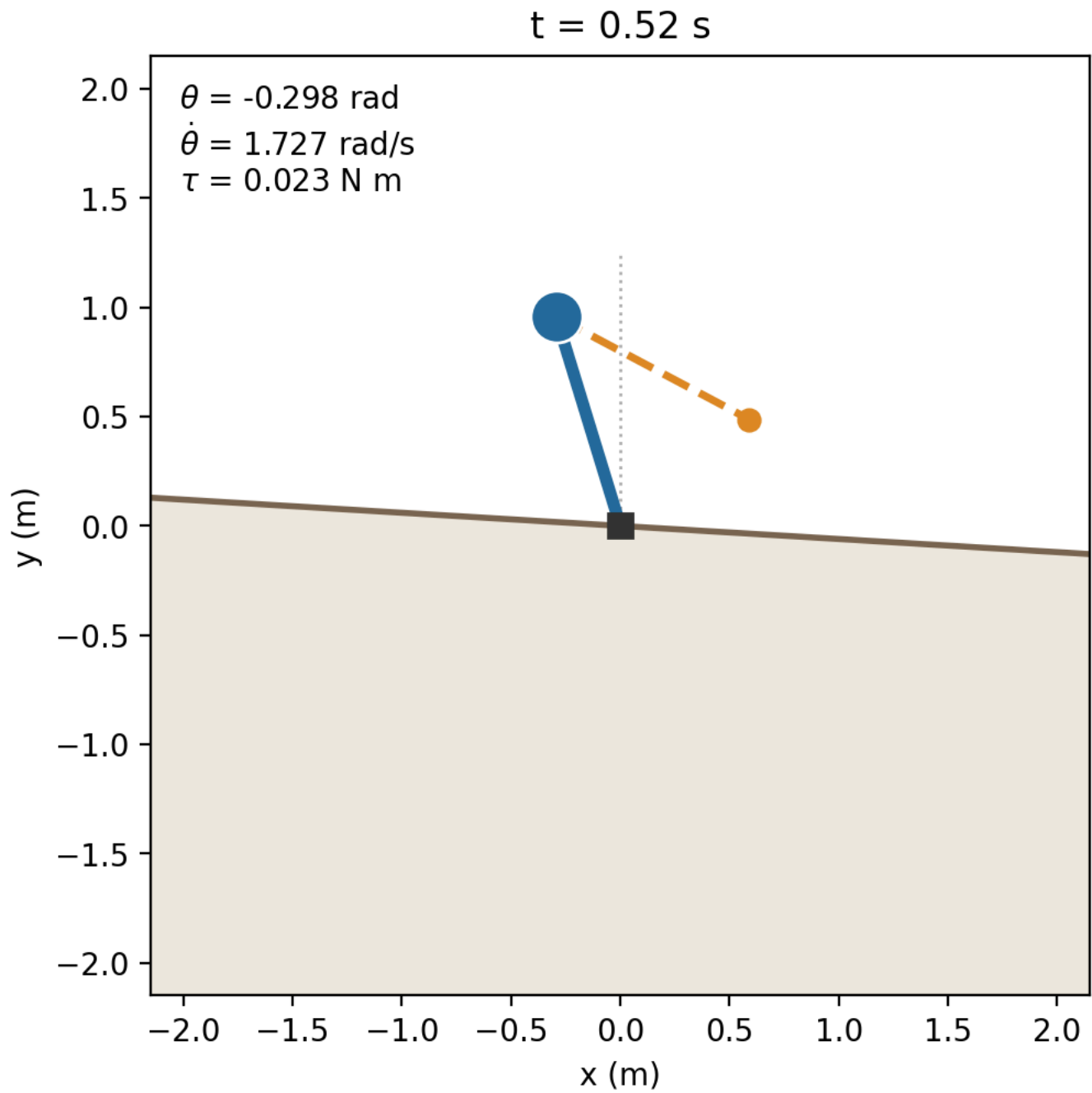
the walker back towards the equilibrium. Vice versa, if $\theta < 0$ then we need $\dot{\theta} > 0$, i.e positive velocity. Thus, this creates a sort of $y = -x$ ROA line as when $x = \theta > 0$ we want $y = \dot{\theta} < 0$ and vice versa. This is seen exactly in the about ROA figure, making sense.

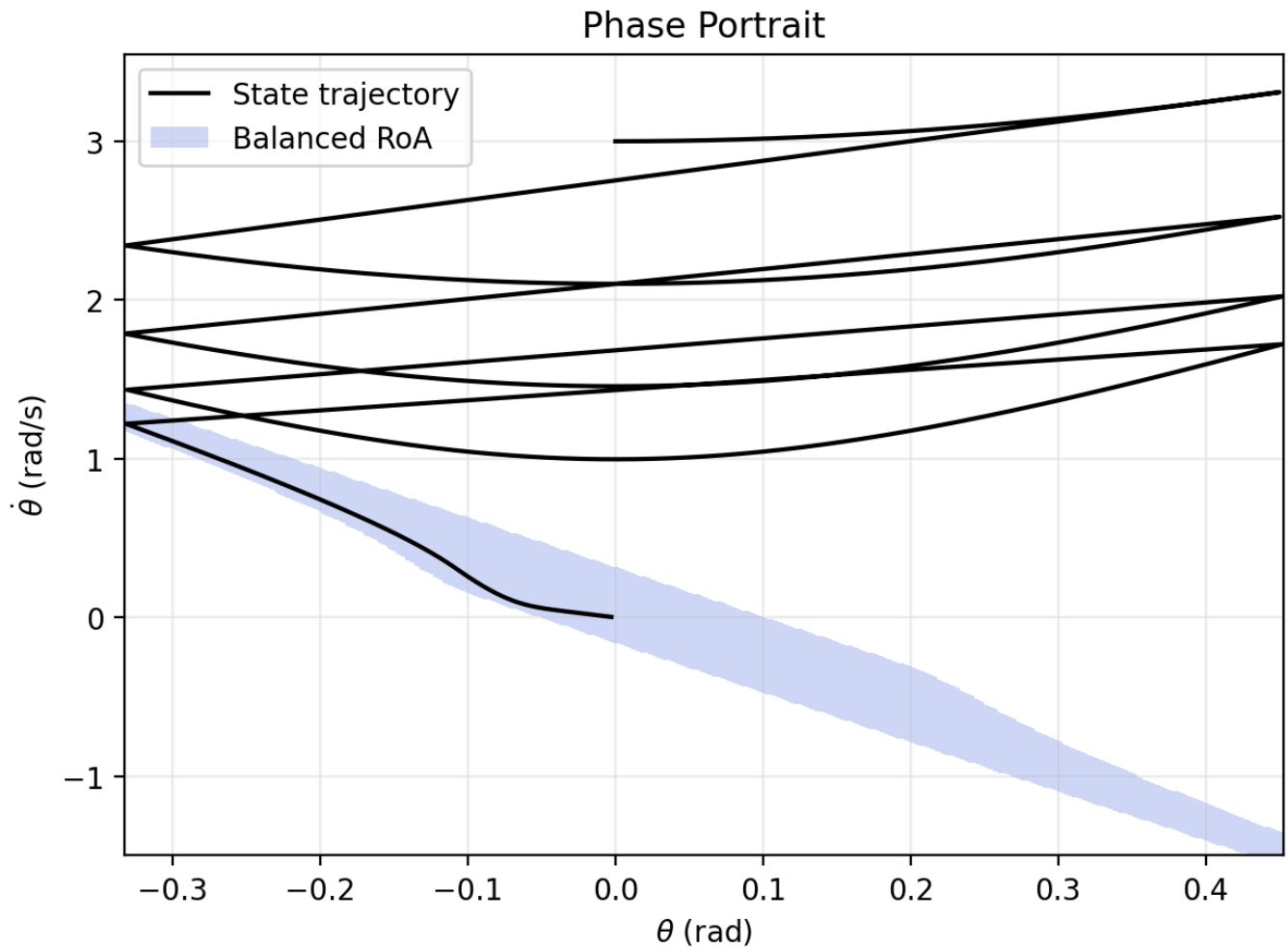
Running a trajectory simulation

Now that we have calculated the ROA, we can use the fact that we are walking down hill to our advantage. When simulating a trajectory, we can let the walker continue walking and at each time step check whether or not we are in the ROA. Once in the ROA, we can apply the required torque to force us to $\theta = \dot{\theta} = 0$. This leaves the ankle controller off just until we need it, making it computationally and realistically effective.

Below are snapshots of a trajectory along with a phase portrait overlaid on top of the ROA. Specifically, the simulation started at $[\theta, \dot{\theta}] = [0, 3]$ and we know that we reach the ROA at 1.6 seconds in and the state we reach the ROA at is $[-0.331690661.22026485]$.



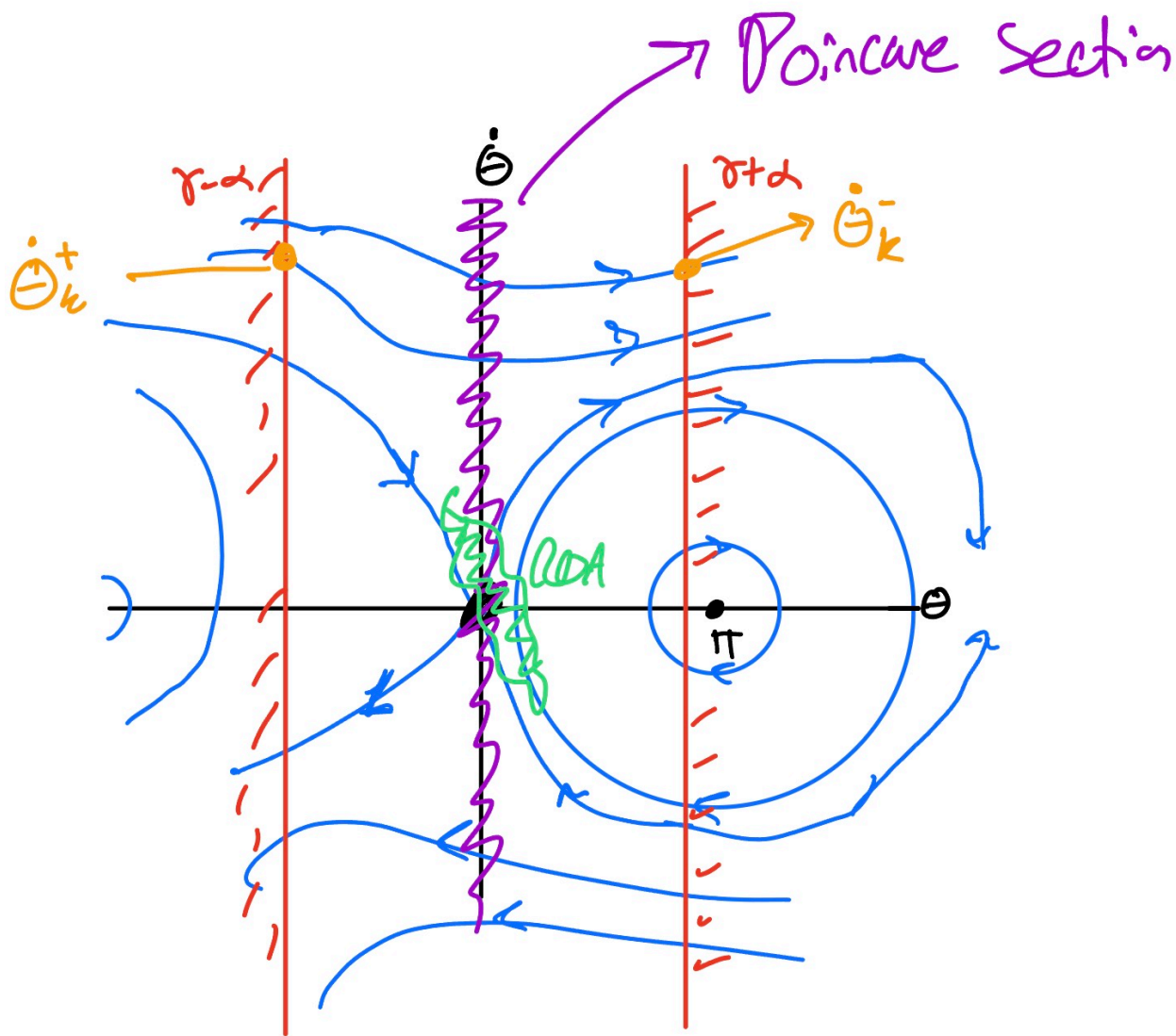




In this phase portrait we clearly see that this walker requires four total steps to reach the ROA (along with approximately 8 hits of the poincare section which we introduce shortly) and we see that once it hits the blue shaded ROA we quickly return to the origin equilibrium of $[\theta, \dot{\theta}] = [0, 0]$. Since we do not turn on the controller until it hits the ROA this is the maximum number of steps it can take before reaching the ROA as we are deliberately not slowing it down until we know we can stablize it quickly.

Poincare Section

From here, we choose a poincare section of $\theta = 0$. This allows us to track everytime the walker crosses over that vertical hump and continues down on its way. The below sketch illustrates our choice:

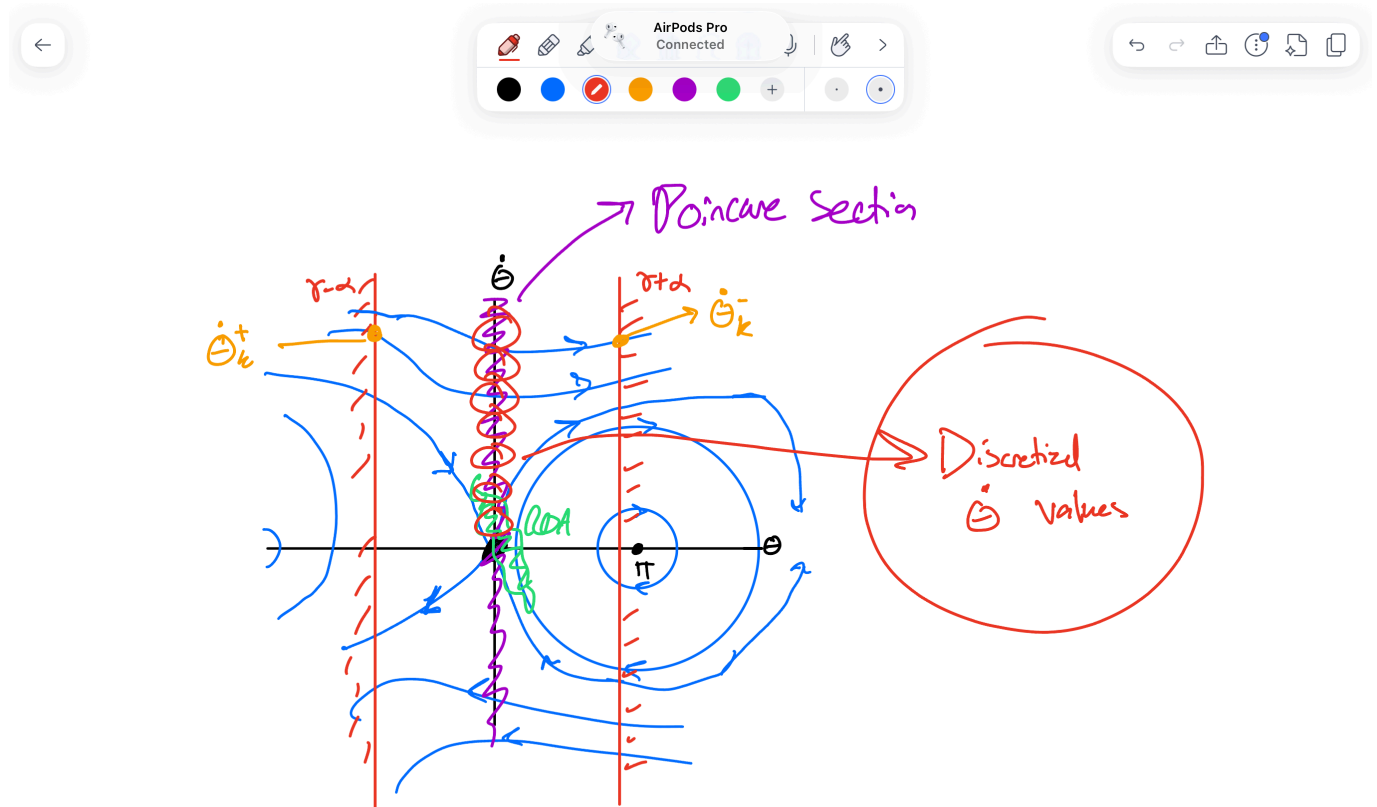


We see my rough sketch of the phase portrait along with the ROA, but more importantly we see how the poincare section will record each time a trajetroy crosses our y axis i.e. theta equal to zero.

This section allows our θ state to be constant, meaning we only have a single dimension of $\dot{\theta}$ to keep track of and store in our trajectories. This makes our life easy for when we want to track which trajectories end up in the ROA as we can simply look at how the velocity changes at each point where our walker's leg is perfectly vertical.

Lookup Table

Intuitively, we think that as we start walking down, we slow down and eventually reach the controller ROA where we turn on the ankle control and send our walker to balancing at $\theta = \dot{\theta} = 0$. However, how do we actually find these trajectories? This is the point of discretizing our space and creating a lookup table using the chosen poincare section as illustrated below:



In the above figure we see that essentially we are looking for where a trajectory hits those circles so we can discretize our state space. With this comes another challenge from the fact that we can control α .

As we discussed in class, this is very variable. Depending on both the step size (α) and the discretization number, we may be able to step past a couple of states or jump further down the vertical poincare section.

Overall, in this assignment we created a lookup table that shows the poincare section measurements. Specifically, it is a $n_{\text{theta_dot}} \times n_{\text{control}}$ sized matrix. For example, consider having the i, j 'th entry of this matrix being 2.5. What that means, is that given the velocity at row i , and the j 'th control input, at the next hitting point of the poincare section the walker will have velocity $\dot{\theta} = 2.5$.

This state action space lookup table allows us to see where trajectories and paths are moving between states. Within my own implementation of this, we swept through initial angular velocities of $[0, \sqrt{2g/l}]$ and swept through the permissible range of the step size α . From here, in the table I also included "flags" that indicate whether this step was in the ROA or if it never hits the poincare section again (i.e. comes to a standstill rest). Specifically, these flags are **ROA** = -100 and **FAILURE** = nan.

A typical lookup table may look something like the following:

$$\mathbf{T}_{20 \times 3} = \begin{bmatrix} -100 & -100 & -100 \\ -100 & -100 & -100 \\ -100 & \text{NaN} & \text{NaN} \\ -100 & -100 & \text{NaN} \\ -100 & -100 & \text{NaN} \\ \vdots & \vdots & \vdots \\ 3.1217 & 2.9083 & 2.6744 \end{bmatrix}, \quad -100 \equiv \text{ROA}, \quad \text{NaN} \equiv \text{FAILURE}. \quad (1)$$

Once again, what this means is that say from our max velocity of $\dot{\theta} = np.sqrt(2g/l) = 4.42$ we can use controller 1 to get to velocity 3.12 within a single step back to our poincare section at $\theta = 0$.

Paths Plotting

To plot the different paths, I did a couple of things. First, I converted the lookup table into a labeled look up table. What this means, is the entries now indicate the closest discretization label rather than the velocity itself. For example, having an entry of 10 in the $[i,j]$ position means that from velocity label i , we can get to the 10th discretized velocity via controller j . This allowed me to better build out the paths using the node labels.

A typical labeled table may look like,

$$\mathbf{L}_{20 \times 3} = \begin{bmatrix} -100 & -100 & -100 \\ -100 & -100 & -100 \\ -100 & 0 & 0 \\ -100 & -100 & 0 \\ -100 & -100 & 0 \\ \vdots & \vdots & \vdots \\ 14 & 13 & 12 \end{bmatrix}, \quad -100 \equiv \text{ROA}, \quad 0 \equiv \text{FAILURE}, \quad (2)$$

where again, we say that 14 in the $[20,1]$ position indicates that from velocity label 20 using controller 1 we can go to velocity label 14 (which as discussed above was velocity 4.42 to 3.12).

From here, we implemented a recursive function to go through and look at every single index of the labeled lookup table and find all paths upstream of it. For example, say we start by going to the $[15, 1] = 10$ entry and see that from velocity 15, using controller 1, I can get to entry 10. Then, I begin a search through the table for any place I can reach 15 and see what different controllers I can use to reach it. From there, at each one of those velocities, I take another step backwards to see where we can reach. Again, I implemented this using a rather fun (in my opinion) recursive search to find all paths.

I finally then stored all of these paths in dictionaries labeled `path_dict` and `control_path_dict` which contained all possible paths through our labeled lookup table. For example, `path_dict[10]` may return something like `[[10, 15, 27], [10, 15, 30], ...]` as it enumerates all possible paths starting from 10. Specifically, I implemented the ability to look at `path_dict["GOAL"]` and `path_dict["FAILURE"]` to see the paths that take us to the GOAL (i.e. into the ROA but we make it point at the GOAL node) and failure states meaning they never hit the poincare section again.

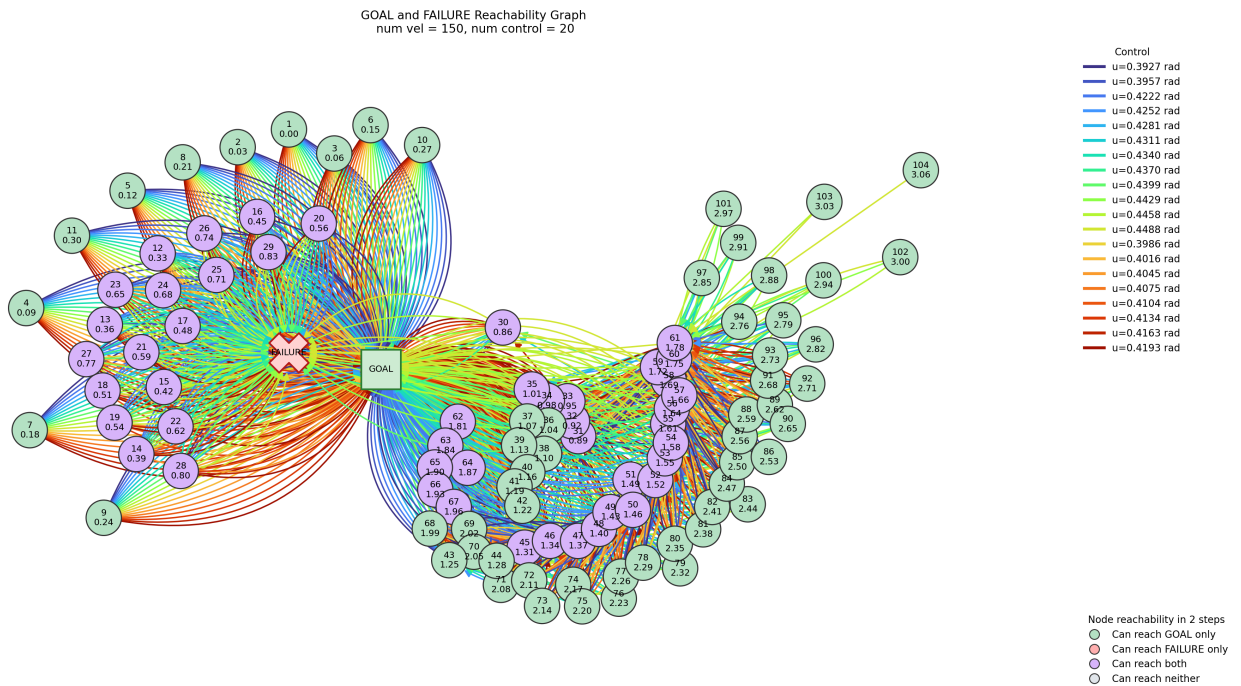
From these dictionaries, I used generative ai to help create beautiful plotting functions that can create visualizations of the graphs we have created. In the following section we will discuss the grid resolution to show how it changes these results.

Grid Resolution

When implementing the grid resolution, we need to think about what it really means. For every grid point we choose along on poincare section, we have designated another node in our graph. Then for every control input we choose to linearly space across the permissible values, we introduce a new edge from each node.

I selected the grid resolution experimentally based on making a graph presentable while also meaningful. However, maybe even more meaningful is the depth chosen. While the code I created can generalize to any depth, it is important to note that we chose a depth of 1-3 for all the following images simply because they get crowded so quickly otherwise.

Lets start wiht some case work. If we choose really large numbers for our $\dot{\theta}$ grid resolution, the graphs not only become useless as they are practically impossible to read, but they also simply loose meaning. The below figure illustrates just how complex this discretization can get. Assume that we want to try to make our discretization as close to continous as possible so we choose these super large numbers. Our velocity changes are simple too small between nodes and the graph becomes way too complex. We are not getting enough bang for our buck and loose out on the computation time for this figure:



Now take the case where the grid resolution is too small, i.e. something like $n_{\dot{\theta}} = 5$. Here, the classification of states as reaching the goal or failure is sensitive to the grid as we see that when we increase our resolution whether or not a state goes to the ROA in one or two or three steps, etc. changes dramatically. The below figure shows this with only 2 control inputs as well. One would notice in the figure below that while we have 5 velocity states, we only see three nodes. This is because only three of the five nodes can actually make it to the GOAL or failure in just two steps. This is losing out on a lot of potential data as we are kinda saying that anything above node 3 ($\dot{\theta} = 2.21$) we cant reach. This is obviously not true as seen in the other figures. Therefore, this is obviously not a good choice.

State transition graph

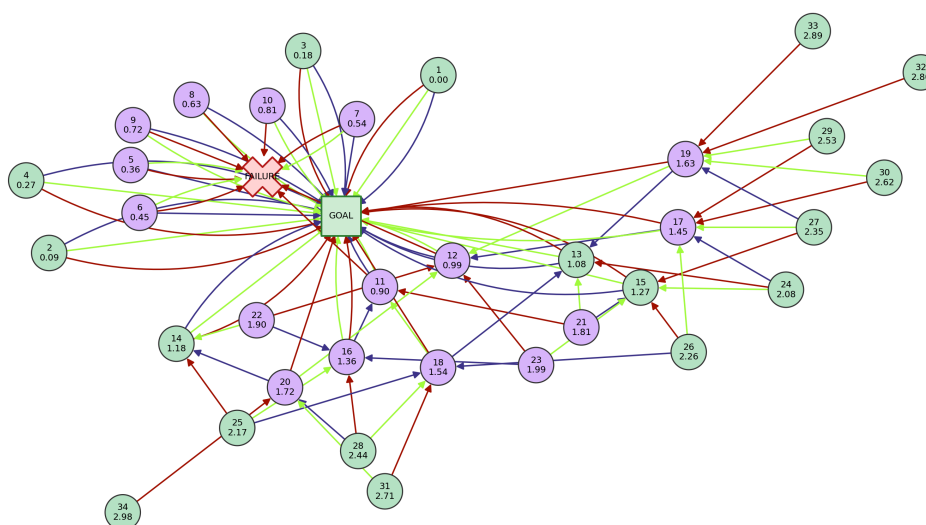


Control
 $u=0.3927$ rad



Node reachability in 2 steps
 ● Can reach GOAL only
 ● Can reach FAILURE only
 ● Can reach both
 ○ Can reach neither

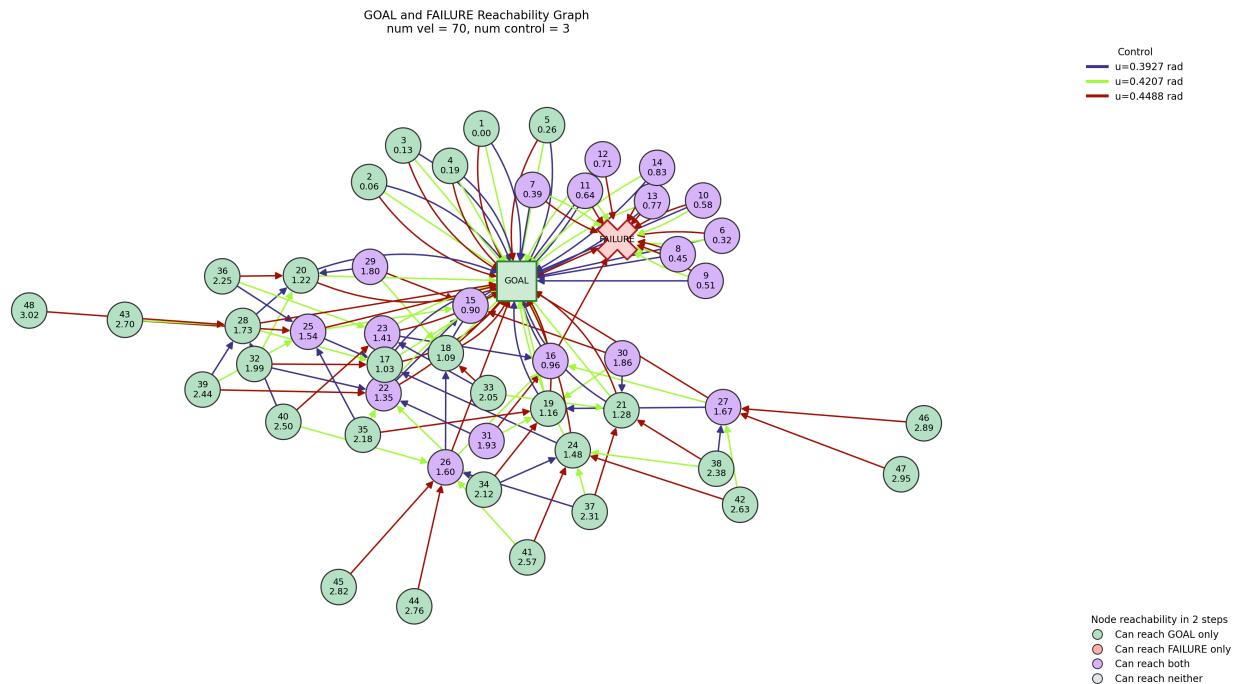
At approximately 70 velocity states, the overall structure of the graph became fairly stable. The way I came to this conclusion was simply trial and error. For example, what I was looking for was at each discretization, what was the highest velocity that could still get to the GOAL within two steps. At $n = 50$ nodes, we see that $\dot{\theta} = 2.98$ could still get there.

GOAL and FAILURE Reachability Graph
num vel = 50, num control = 3

Control
 $u=0.3927$ rad
 $u=0.4207$ rad
 $u=0.4488$ rad

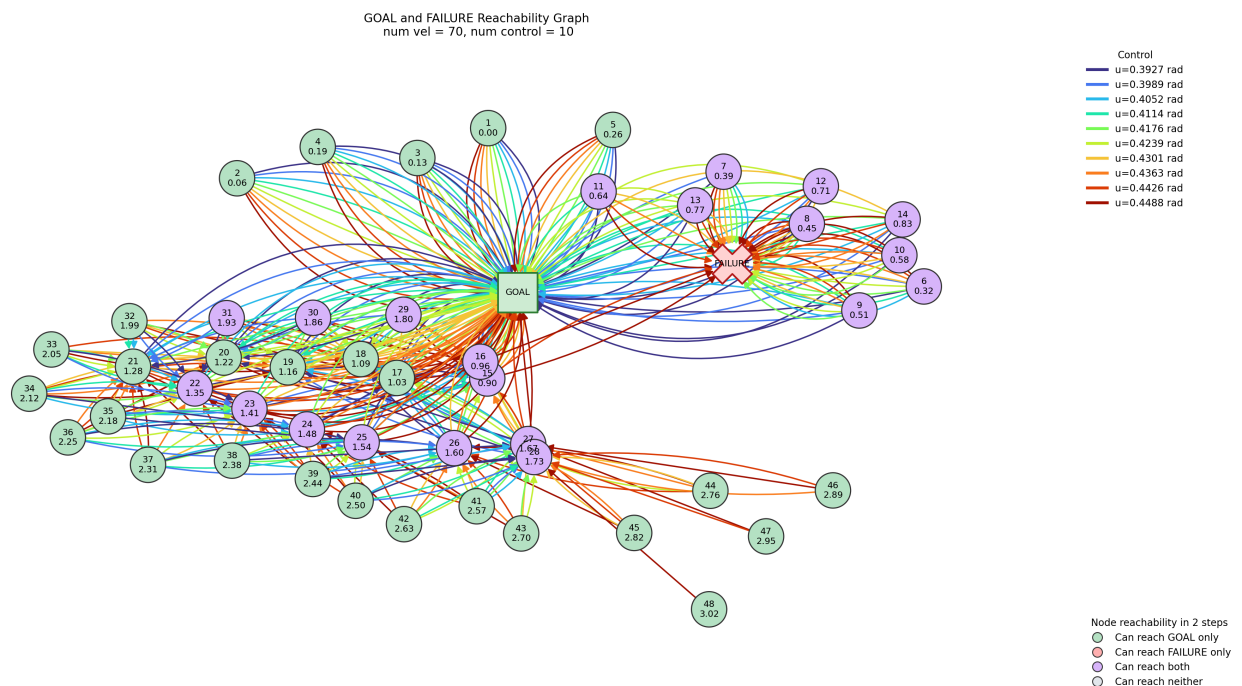
Node reachability in 2 steps
 ● Can reach GOAL only
 ● Can reach FAILURE only
 ● Can reach both
 ○ Can reach neither

However, at $n = 70$ nodes we see that we have increased our top velocity to node 48 at $\dot{\theta} = 3.02$ as seen below on the right hand side:



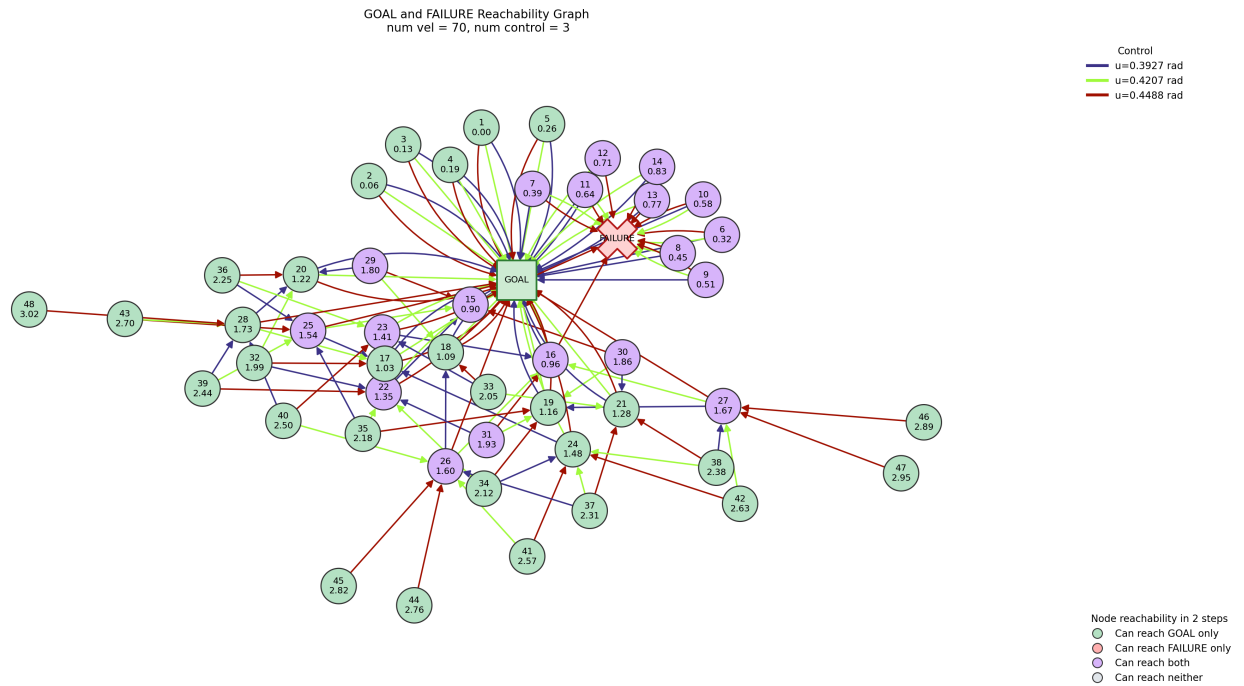
However, at $n = 100$ nodes we have only increased up to a velocity of $\dot{\theta} = 3.04$ a very minimal increase. This increase is not really worth the complexity of the graph and the computation time. Therefore, we stick with $n = 70$ as our grid resolution, although a grid of $n = 50$ really does the trick.

Now the question is how many control inputs should we use. Overall I ended up choosing 3 different discretized inputs. Below we see that even at 10 different inputs, we still only get a max velocity of 3.04 that we can stabilize in two steps. This means that a better continuous control over the step size α really doesn't help us stabilize any quicker here:



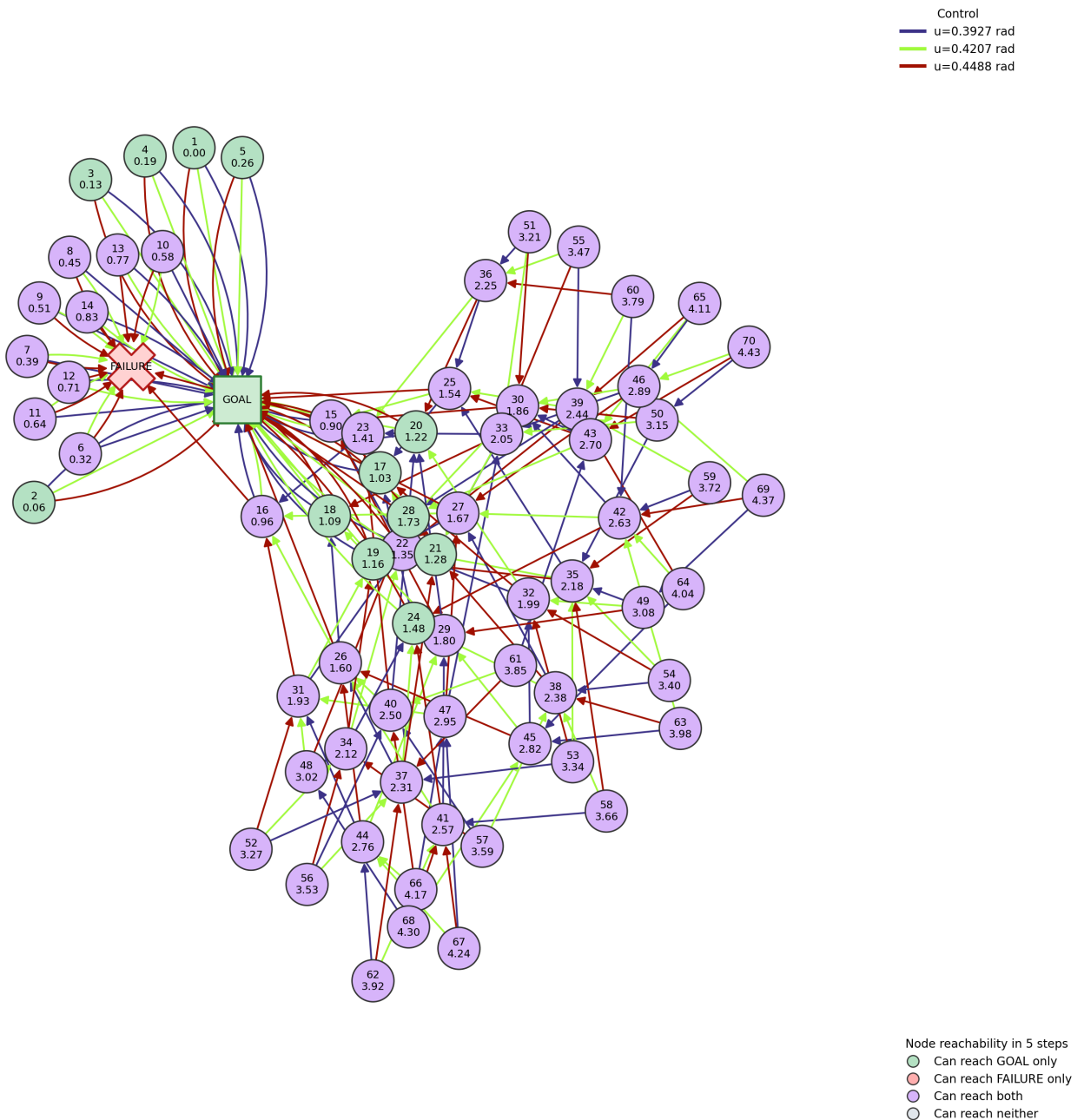
Therefore, the final grid resolution I chose was 70 velocity points and 3 alpha control points. I realize that this resolution is still probably too fine and provides a lot of detail but I was able to run it rather quickly and so I don't see many downsides to having too much detail.

The below figure shows the final result of our GOAL and FAILURE graph, illustrating how we can step towards the goal of upright balance.



Finally, I also want to point out here that although we discretize to 70 velocity states, we don't see 70 nodes. This is because many nodes cannot reach the ROA or FAILURE within only two steps. This is why I want to finally illustrate the entire graph rather than just the paths to GOAL or FAILURE. The below figure shows the entire graph for 70 velocity states and 3 control inputs, along with an increased depth of paths of length 5!

GOAL and FAILURE Reachability Graph
num vel = 70, num control = 3



Here we see that all 70 nodes are present and fully accounted for. However, we still see that the main sink of our network is the GOAL node showing that most paths will end up letting us walk to that balancing upright equilibrium.

Conclusion

In conclusion, we were able to balance the walker from almost any state all by just letting it walk on its own and then catching it using our torque controller once it hits the ROA. We see from the graph analysis that the step size angle α can really change how quickly we can get to the goal along with the ability to get out of states that sometimes may go to failure. This is possibly the most interesting case as we see in those images that many of the nodes are colored purple, meaning that they can go to the goal and failure depending on the step size chosen! Thus, this is the most valid piece of evidence we have that this step size matters! But overall we can see that we have created a valid way of looking at the different paths we can take and if we were implementing RL here we would want to choose the paths that get us to the goal quickly and effectively while avoiding failure.